NotifyEngine

Sprint 1 - Core Notification Pipeline

Algo-Rhythms | CS 691 Capstone | Pace University

March 2026

Agenda

01 Team Roles & Responsibilities

02 Improvements from Professor Feedback

03 Project Description & Team Agreement

04 Personas & MVP Definition

05 Technologies & Algorithms

06 Architecture & Diagrams

07 Product Backlog & Sprint 1 Backlog

08 Test Cases & Sprint Results

09 Metrics: Velocity, Burndown, Ratio

10 Retrospective & Sprint 2 Planning

11 Project Demo & GitHub

NotifyEngine

TEAM ROLES

Mohammed N. Yusif

Security Engineer/ Project Manager

Krish Dhorajiya

AI Engineer / Tester

Rhythm Patel

AI Engineer / Developer

TEAM ROLES

Salvatore Ardisi

SCRUM Master / Developer

Dhawalshree Mengane

Data Analyst / Developer

Zeeya Ramani

Data Analyst / Developer

Improvements from Professor Feedback

The professor advised our team to give individual introductions while presenting so that each member is properly introduced in the presentation.

The recorded video had low resolution when posted, so we need to take care to ensure better video quality and resolution in future recordings.

Project Description

WHAT
An ML-powered notification delivery service that learns from user behavior and automatically routes notifications to each user's most engaging channel (sms, email, push and in app channels).

WHO
For developers and product teams who send high-volume user notifications across different channels

WHY
Unlike static notification tools like Courier, Knock, or Novu that use fixed rules, and enterprise platforms like Braze or MoEngage that are expensive, NotifyEngine offers a simple developer-first API, per-user channel routing that continuously improves with every notification sent.

BENEFITS
Higher notification engagement rates, lower SMS spend, automatic failover during provider outages, and zero manual routing logic to maintain.

Team Working Agreement

- Meetings: Tuesdays & Thursdays via Zoom + async Slack standups 3x/week
- Deadlines: Individual work due 48 hours before sprint deadline for integration
- Communication: All project discussion in Slack (not WhatsApp/text)
- Accountability: Missed deadlines documented; escalation path defined
- Code: All changes via pull requests - no direct pushes to main/develop
- Quality: npm audit + security scan before every PR merge

Personas - Customer Personas

Sarah Kim
Backend Developer

Series A SaaS (75K users)

Sarah is responsible for integrating email, SMS, and push providers. She currently manages SendGrid, Twilio, and Firebase separately. When a provider fails, she must manually reroute notifications.

Pain Points

- Multiple provider integrations
- No automatic fallback during outages
- Manual rerouting during incidents
- Complex maintenance overhead

Maps to NotifyEngine

- Unified API
- Circuit breaker pattern
- Automatic channel fallback
- Reduced engineering overhead

Daniel Ross
Product Manager

Fintech app (500K users)

Daniel notices engagement rates declining over time. Email open rates have dropped significantly, but routing logic hasn't been updated in over a year.

Pain Points

- Declining engagement rates
- Static routing rules
- No behavioral insights
- No visibility into channel performance

Maps to NotifyEngine

- XGBoost adaptive routing
- Real-time engagement dashboard
- Model explainability
- Automatic learning from outcomes

Amanda Lopez
Growth Lead

E-commerce platform (2M users)

Amanda spends heavily on SMS notifications because that has always been the default channel. She suspects cheaper channels could perform similarly.

Pain Points

- High SMS delivery cost
- No cost optimization strategy
- Engineering dependency for testing
- No performance breakdown by channel

Maps to NotifyEngine

- Cost-aware routing
- Channel performance analytics
- Usage-based pricing alignment
- Zero manual configuration

MVP Definition

1. POST /v1/notifications with API key auth
2. Zod validation + notification record created
3. Job enqueued to BullMQ, 202 Accepted returned
4. Worker picks up job, selects channel
5. Email delivered via Nodemailer (Mailpit)
6. Delivery outcome recorded in PostgreSQL
7. GET /v1/notifications/:id returns full status

Sprint 1 Scope

36 Story Points

4 Epics + ML Prep

14 Acceptance Tests (MVP)

202 Async Response Code

Technologies

Backend

Node.js + TypeScript

Express

PostgreSQL 16

Redis 7 + BullMQ

Zod Validation

Pino Logging

ML / Data

Python 3.11 + FastAPI

XGBoost (scikit-learn)

pandas

Synthetic Data Gen

Frontend

React 18 + Vite

TanStack Query

Socket.IO (real-time)

Tailwind CSS

DevOps / Security

Docker Compose

Turborepo Monorepo

GitHub Actions CI

Helmet + CORS

Prometheus + Grafana

Algorithms

XGBoost (Gradient Boosting)

ML Classification

Supervised ML model trained on notification delivery outcomes (delivered, opened, clicked, dismissed, ignored). Input features: channel type, time of day, day of week, hours since last notification, notification count, user engagement history, channel specific open rates. Output: probability of engagement per channel (highest probability wins)

Epsilon-Greedy Exploration

Exploration Strategy

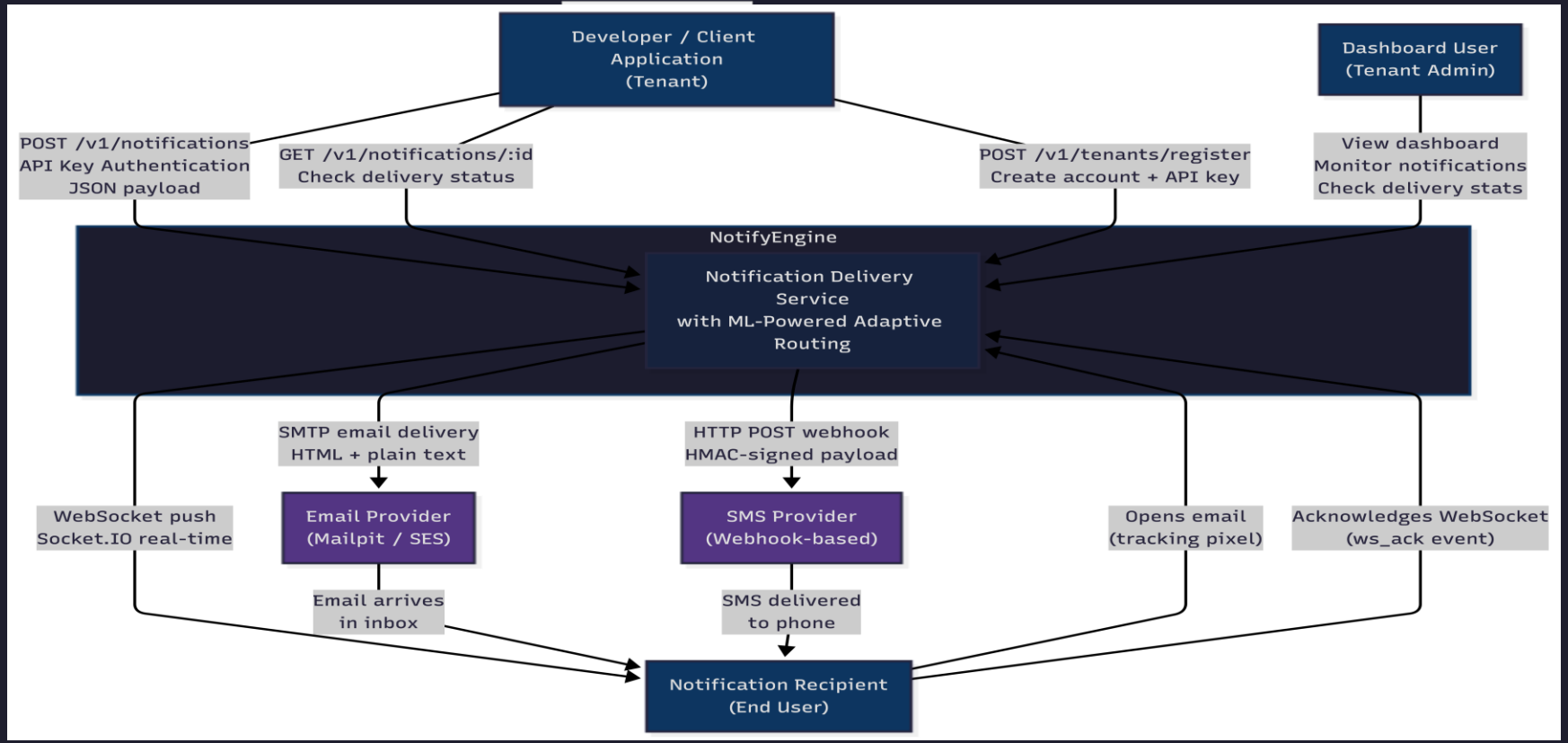
90% of the time (exploit): route to the channel the model predicts will perform best. 10% of the time (explore): randomly try a different channel to discover new patterns. Prevents the model from getting stuck (if a user changed behavior, exploration detects it).

Circuit Breaker Pattern

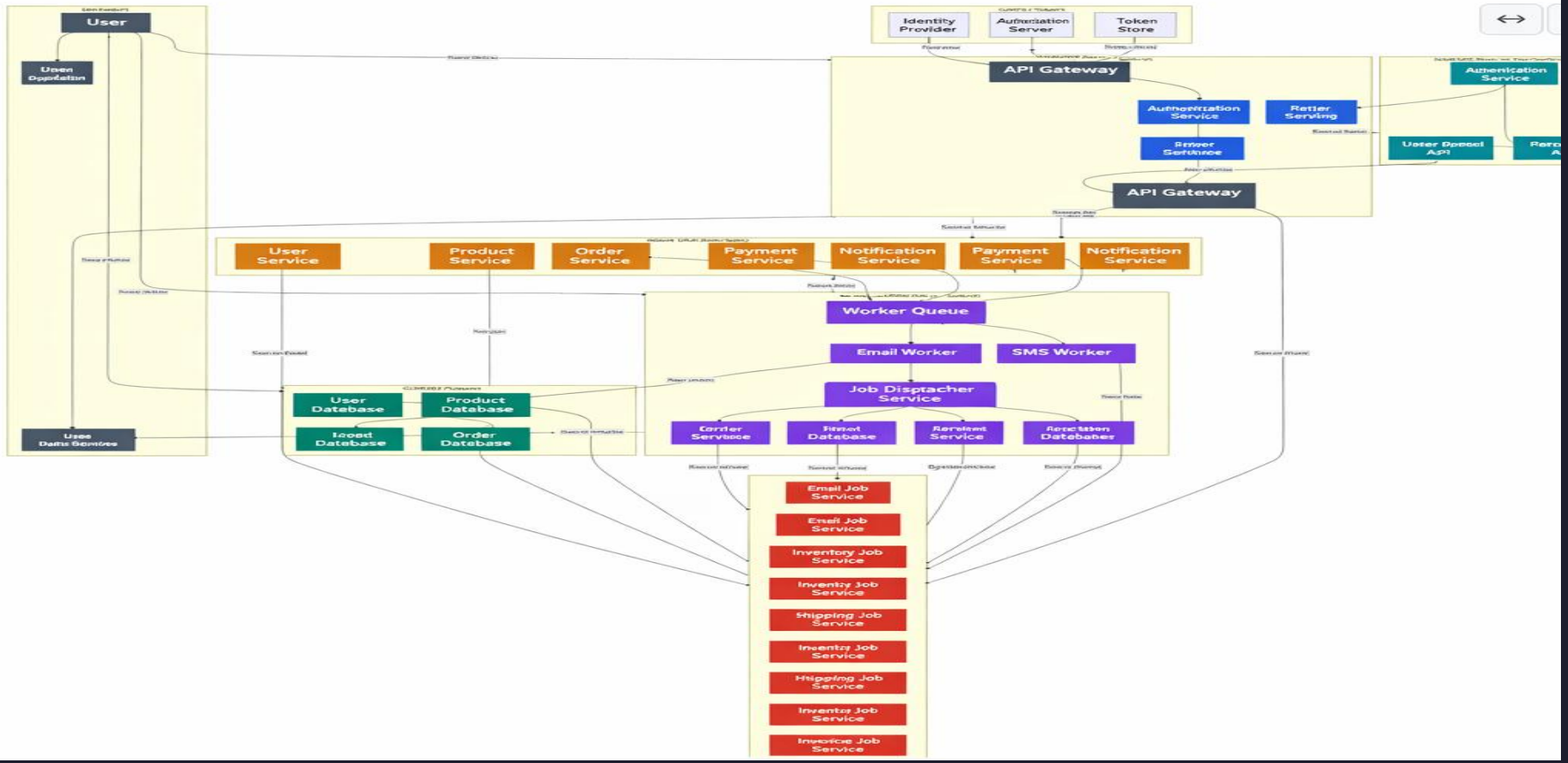
Fault Tolerance

Monitors delivery failures per provider in real-time. If failure rate exceeds threshold -> circuit opens -> traffic reroutes to next best channel. Automatically retries the failed provider after a cooldown period.

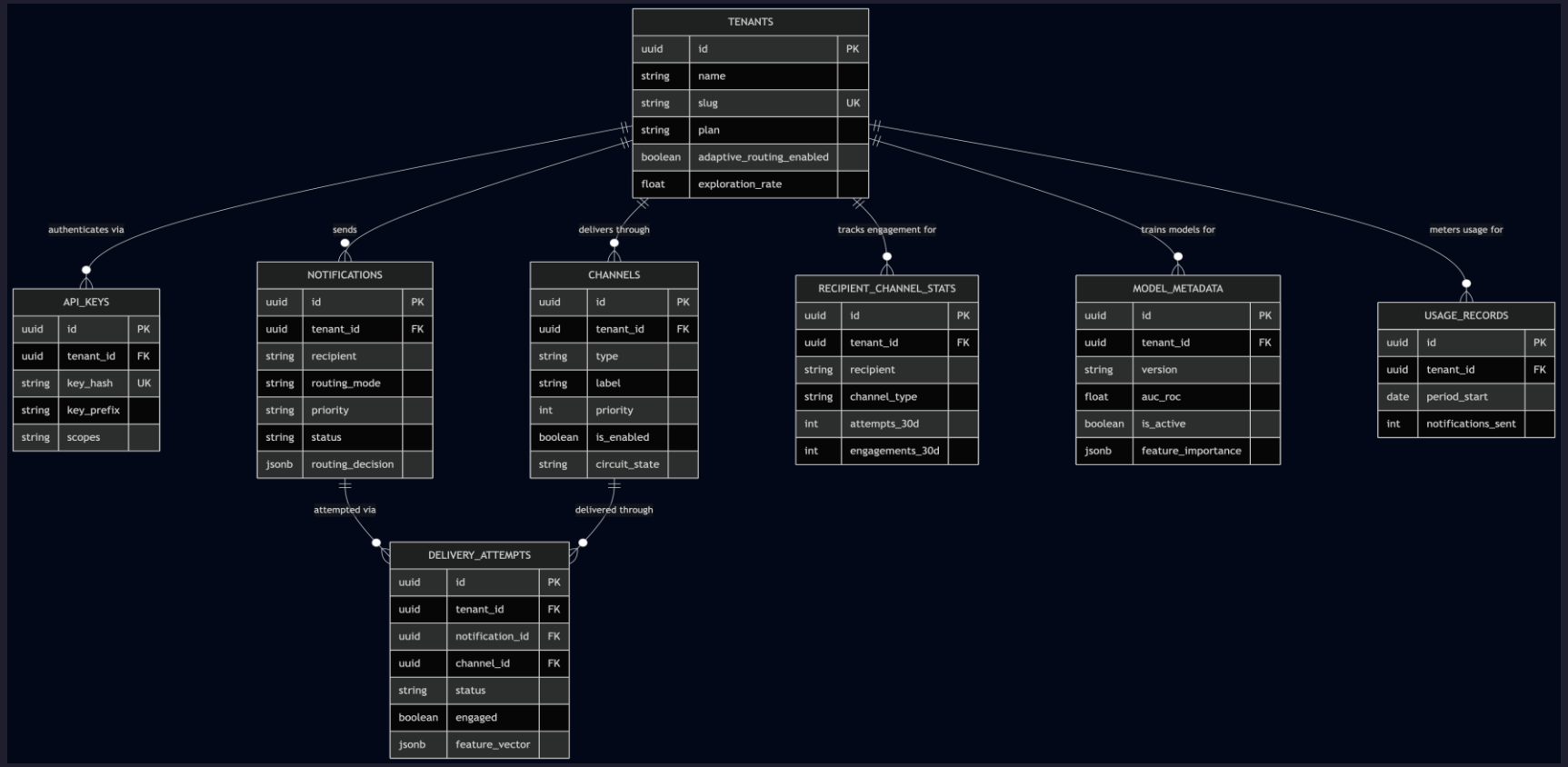
Context Diagram



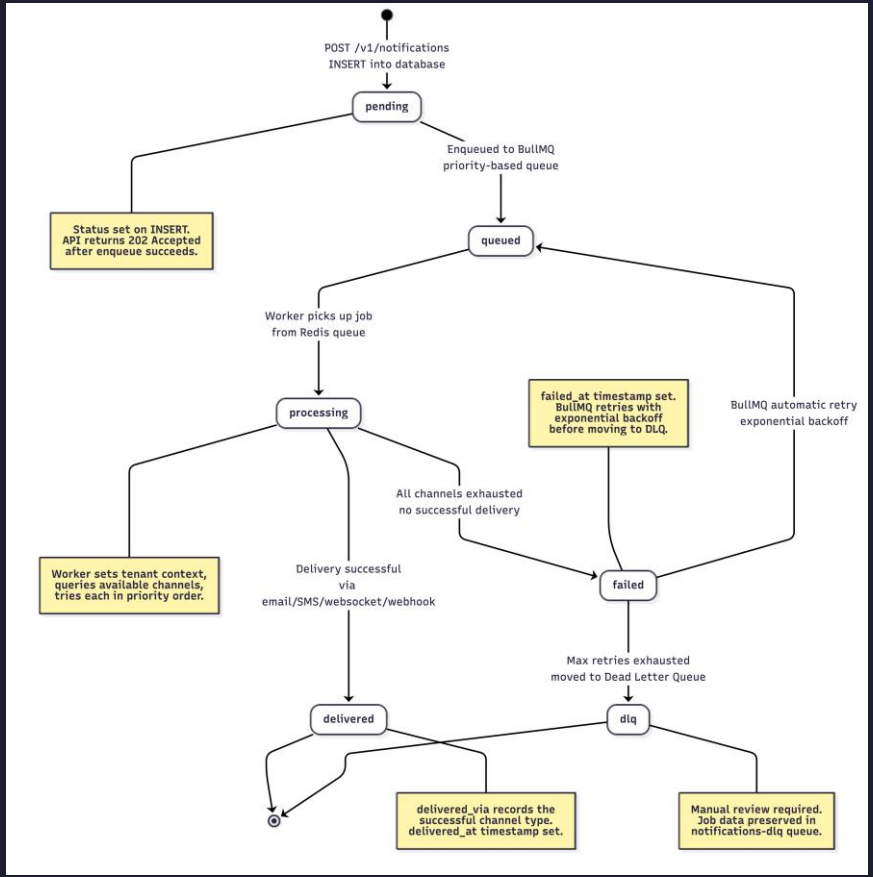
Architecture Diagram



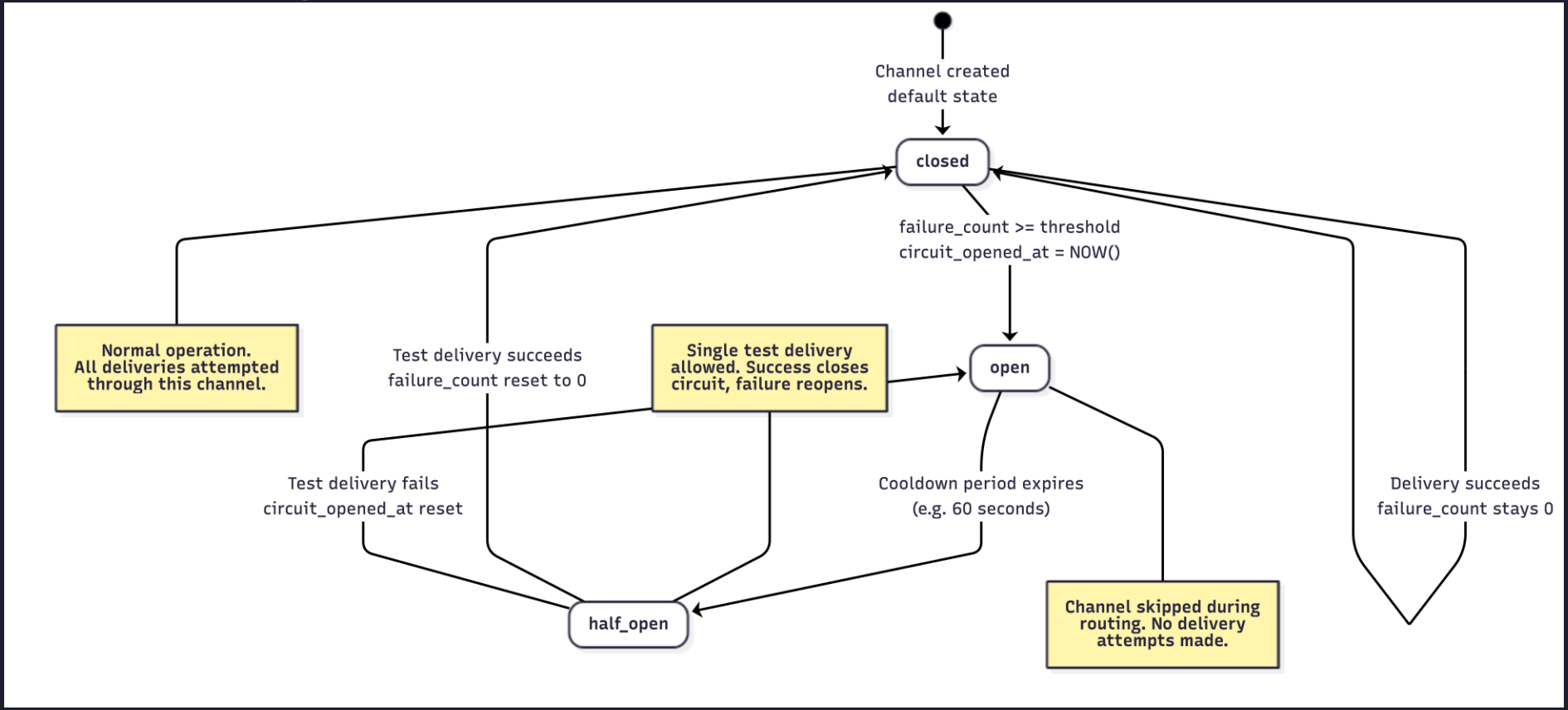
Conceptual Diagram



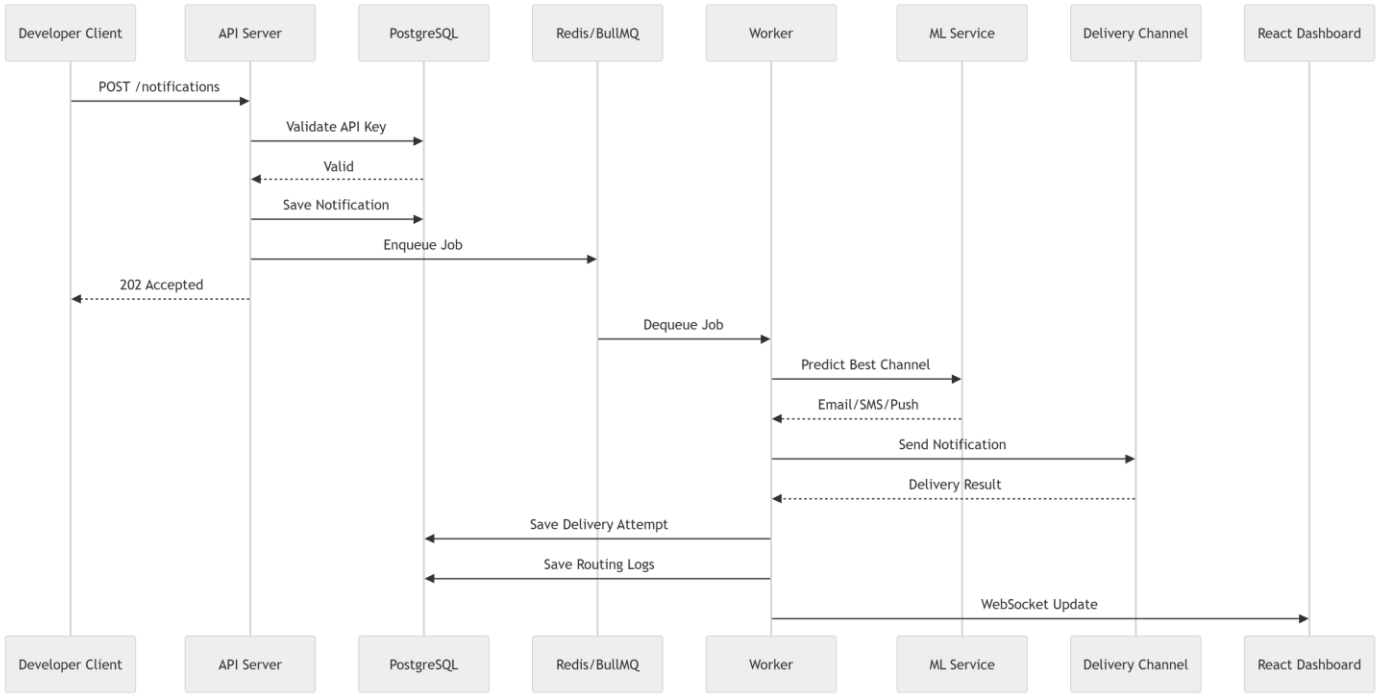
State Diagram Notification Lifecycle



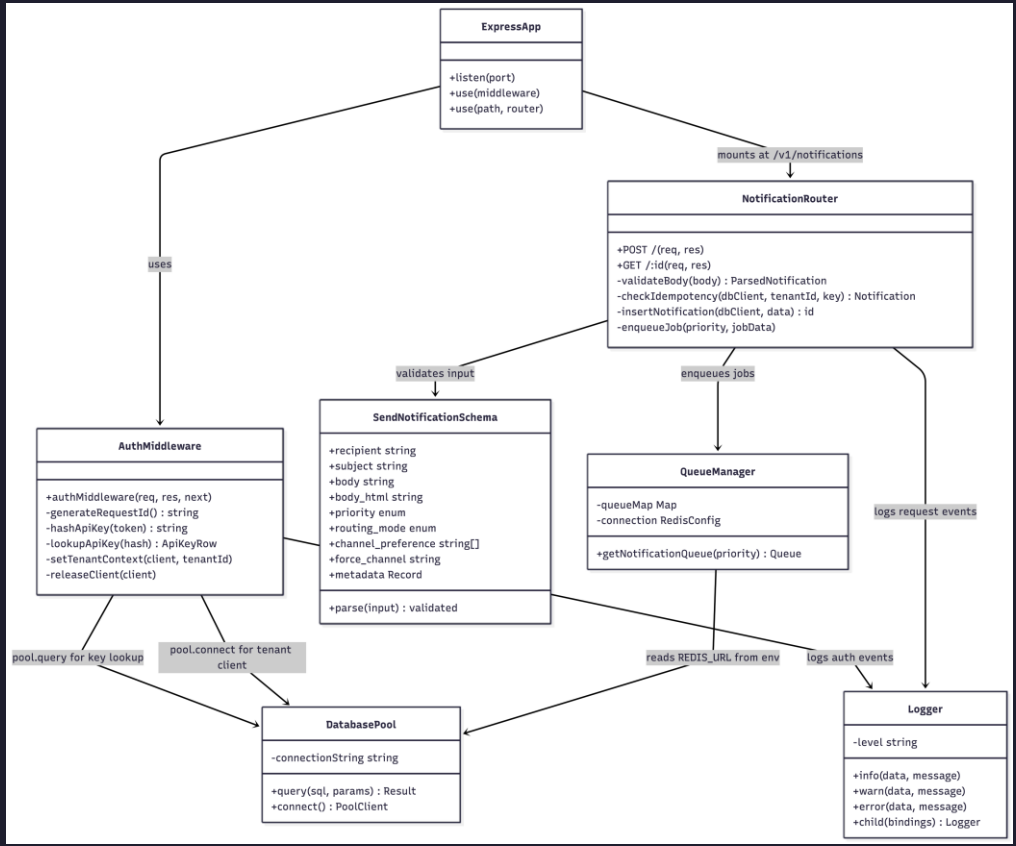
State Diagram Circuit Breaker



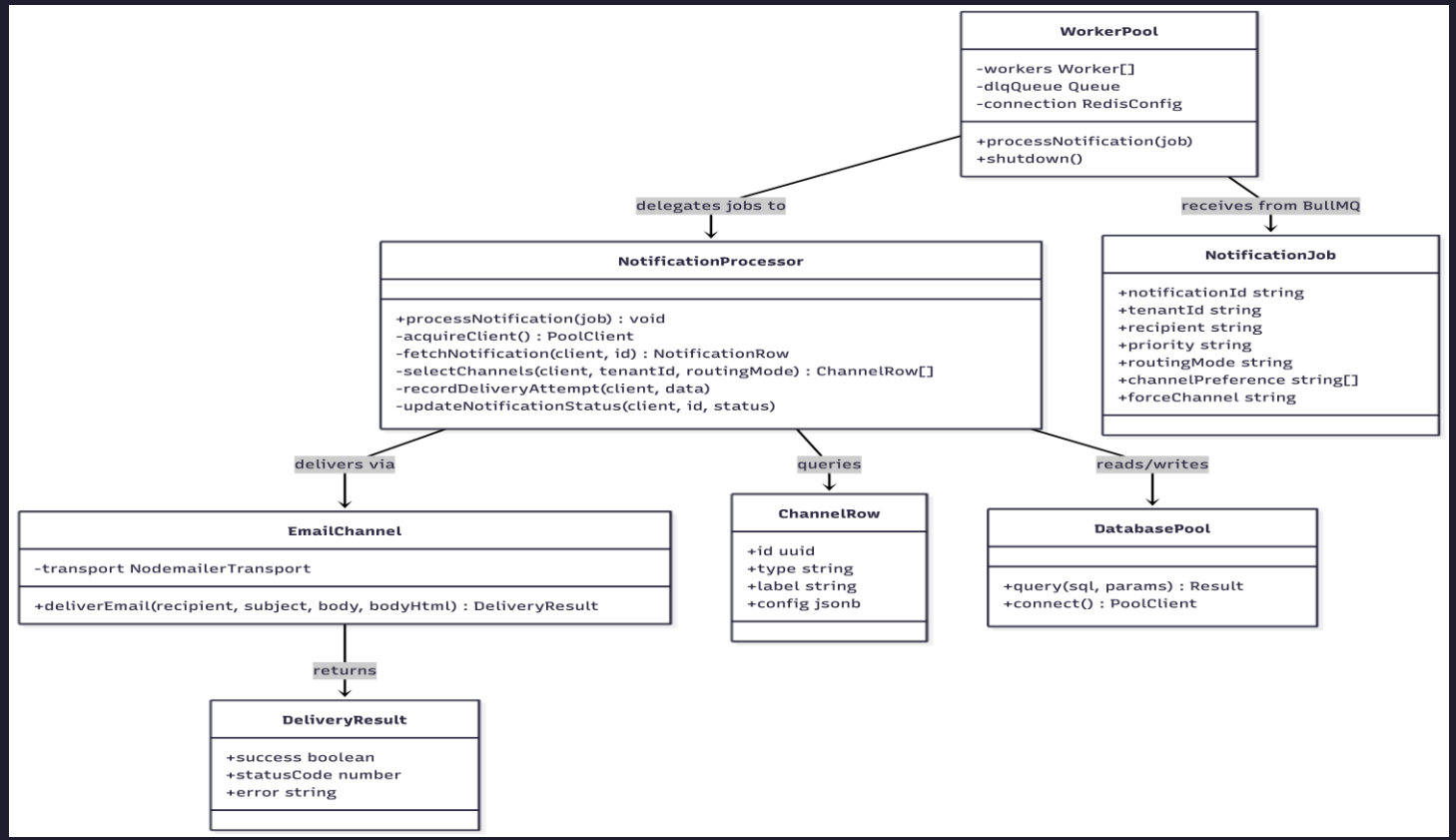
Sequence Diagrams



Class Diagram Api Server



Class Diagram Worker Service



Product Backlog

ID	User Story	Acceptance Criteria	Feature	Component	Story Points	Priority
PB-01	As a developer, I want to register my company as a tenant so that I can get an API key to use NotifyEngine	- Tenant registers via API with company name and email - System generates unique API key (SHA-256 hashed) - Duplicate registration returns error - Response includes tenant ID and API key (shown once)	Tenant Registration	Multi-Tenant API	3	High
PB-02	As a developer, I want to send a notification via a single API call so that I don't have to manage multiple providers	- POST /v1/notifications accepts recipient, message, routing_mode - Request validated via Zod schema - API returns 202 Accepted with notification ID - Invalid requests return 400 with field-level errors - Notification job enqueued in BullMQ	Unified Notification API	Multi-Tenant API	5	High
PB-03	As a developer, I want my tenant's data isolated so that my customers' notifications are never visible to other tenants	- Every DB query scoped by tenant_id - PostgreSQL Row-Level Security policies enforced - Cross-tenant access returns 403 - Dashboard shows only tenant's own data	Multi-Tenant Isolation	Multi-Tenant API	5	High
PB-04	As a developer, I want notifications processed asynchronously so that the API responds fast	- BullMQ job created for every notification - Worker processes jobs in background - Failed jobs retry up to 3 times - Job priority: critical > transactional > marketing	Async Job Queue	Infrastructure	5	High
PB-05	As a developer, I want notifications delivered via email so that I can verify the system works end-to-end	- Worker delivers email via Nodemailer to Mailpit (local SMTP) - Email arrives in Mailpit inbox - Delivery outcome (success/failure) recorded in database - Delivery timestamp logged	Email Delivery	Notification Engine	3	High

Product Backlog

PB-05	As a developer, I want notifications delivered via email so that I can verify the system works end-to-end	- Worker delivers email via Nodemailer to Mailpit (local SMTP) - Email arrives in Mailpit inbox - Delivery outcome (success/failure) recorded in database - Delivery timestamp logged	Email Delivery	Notification Engine	3	High
PB-06	As a developer, I want to check notification status via API so that I can track what happened	- GET /v1/notifications/:id returns full status - Response includes: channel used, delivery status, timestamps - Returns 404 for unknown IDs - Scoped to requesting tenant only	Status Endpoint	Multi-Tenant API	3	High
PB-07	As a developer, I want notifications routed using a static fallback order so that delivery always has a baseline	- Default order: Email → Push → SMS - If preferred channel unavailable, fallback to next - Routing decision logged with reason - Used when ML model not available	Static Routing Baseline	Notification Engine	3	High
PB-08	As a developer, I want all incoming data validated so that bad data never enters the system	- Zod schema validates all POST /v1/notifications fields - Missing required fields return 400 with field-level errors - Malformed JSON returns 400 - Validation happens in middleware	Input Validation	Security	2	High
PB-09	As a developer, I want the entire system to run with one command so that the team can develop consistently	- docker-compose up starts all services - Services: API, Worker, PostgreSQL, Redis, Mailpit, Dashboard - No manual DB setup required - Environment variables via .env.example - README documents setup steps	Docker Compose Setup	DevOps	2	High
PB-10	As a developer, I want a basic dashboard showing my notification activity so that I have visual proof the system works	- Dashboard shows total notifications sent - Recent notification log table with status - Data fetched from API endpoints - Visible at localhost:5173	Basic Dashboard	Dashboard	5	Medium

Sprint 1 Backlog

36 Story Points | 4 Epics + ML Prep

ID	Story	Points	Owner
PB-09	Docker Compose + Schema Migrations	5 SP	Salvatore
PB-03	Row-Level Security Policies	3 SP	Salvatore
PB-08	Zod Validation Middleware	2 SP	Mohammed
PB-01	Tenant Registration + API Key Auth	3 SP	Rhythm, Krish, Mohammed
PB-02	POST /v1/notifications Endpoint	5 SP	Rhythm, Krish
PB-04	Async BullMQ Processing	5 SP	Salvatore
PB-07	Static Routing Logic	3 SP	Salvatore
PB-05	Email Delivery (Nodemailer/Mailtrap)	3 SP	Salvatore
PB-06	GET /v1/notifications/:id Status	2 SP	Rhythm, Krish
PB-10	Dashboard (React SPA)	5 SP	Dhawalshree, Zeeya

Sprint 1 - Acceptance Criteria

PB-09	Migrations run idempotently; all 9 tables created; Docker stack healthy
PB-03	Tenant A cannot read Tenant B data; RLS enforced via session variables
PB-08	Invalid payloads return 400 with field-level errors; valid payloads pass through
PB-01	POST /v1/tenants returns API key; key is SHA-256 hashed in DB
PB-02	POST /v1/notifications returns 202 + notification ID; record in DB
PB-04	Job appears in BullMQ; worker processes within 5s; status updated
PB-07	Channel selected by priority order; logs show selection reason
PB-05	Email arrives in Mailtrap; delivery_attempt recorded with timestamps
PB-06	GET returns notification + delivery attempts + channel + timing
PB-10	Dashboard shows stats cards + notification table from API; auto-refreshes every 30 seconds

Test Cases - Sprint 1

Test Cases — Sprint 1

Test Case	Test Case Title	Step Description	Expected Input	Expected Output	Status	Tester
TC-01	Schema & RLS Integration	Run migrations. Then verify all 9 tables & constraints.	npm run migrate	37 tests pass; 9 tables created; Docker healthy.	Pass	Salvatore
TC-02	API Key Authentication	Call /v1/tenants with valid key. Then call with invalid key.	Header: x-api-key: [key]	Valid: 200 OK. Invalid: 401 Unauthorized.	Pass	Krish/ Rhythm
TC-03	Zod Schema Validation	Submit POST with missing required fields.	Payload: {"msg": "test"} (missing recipient)	400 Bad Request + field-level errors.	Pass	Mohammed
TC-04	Notification Creation	Submit valid POST /v1/notifications request.	Payload: {"to": "user1", "msg": "Hi", "mode": "email"}	202 Accepted + Notification ID; record in DB.	Pass	Salvatore
TC-05	Multi-Tenant Isolation	As Tenant A, query data belonging to Tenant B.	GET /v1/notifications/[Tenant_B_ID]	Query returns 0 rows or 403 Forbidden.	Pass	Rhythm/ Krish
TC-06	BullMQ Processing	Enqueue notification. Monitor worker time.	Valid POST Trigger	Job visible in BullMQ; Status updated in < 5s.	Pass	Salvatore
TC-07	Email Delivery (Mailpit)	Trigger email & verify delivery in Mailpit.	Valid Email Trigger	Email in Mailpit; delivery_attempt in DB.	Pass	Salvatore
TC-08	Status Endpoint Verification	GET request for specific notification ID.	GET /v1/notifications/:id	JSON with channel, status, and timing logs.	Pass	Rhythm/Krish
TC-09	Cursor Pagination	Request notifications with limit=5 and check cursor.	GET /v1/notifications?limit=5	Returns 5 records + valid nextCursor in response.	Pass	Salvatore
TC-10	Notification Summary	Fetch summary report and validate delivery counts.	GET /v1/notifications/summary	Delivered/failed/queued counts match DB records.	Pass	Rhythm/Krish
TC-11	Tracking Pixel	Hit tracking pixel URL and verify engagement flag.	GET /v1/engagement/track?nid=	Returns 200 image/gif; sets engaged=true in DB.	Pass	Zeeya/ Dhawalshree
TC-12	Idempotency Key	Send same notification twice with identical Idempotency-Key.	POST with header Idempotency-Key: [same-key]	Second call returns 200 with original notification ID.	Pass	Mohammed
TC-13	Duplicate Tenant	Re-register same company that already exists.	POST /v1/tenants with existing company name	Returns 409 Conflict; no duplicate record created.	Pass	Salvatore
TC-14	Invalid UUID	GET notification with a non-UUID path parameter.	GET /v1/notifications/not-a-uuid	Returns 400 Bad Request; UUID validation error.	Pass	Krish/ Rhythm

* Highlighted rows (TC-09 to TC-14): updated / newly added test cases

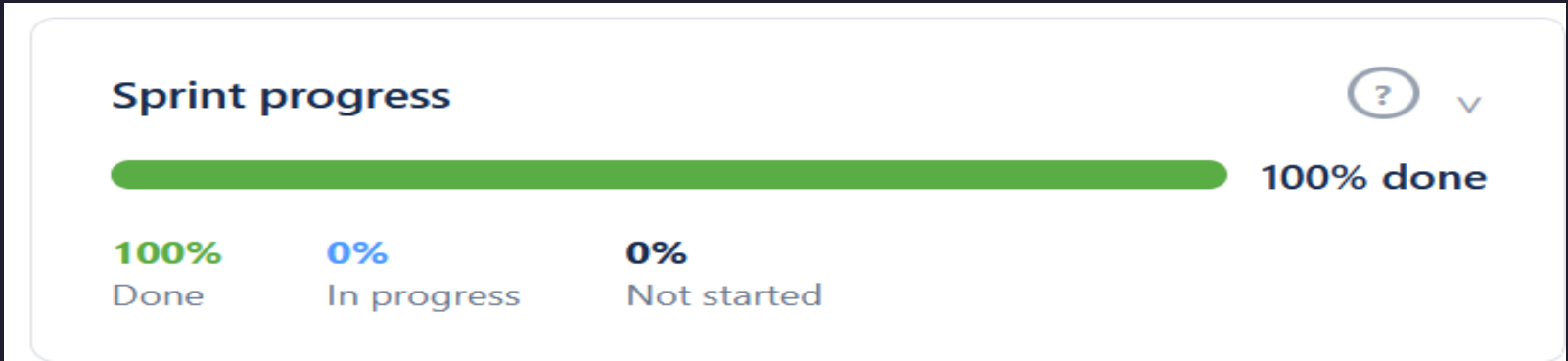
Stories Completed

36 Story Points | 4 Epics + ML Prep

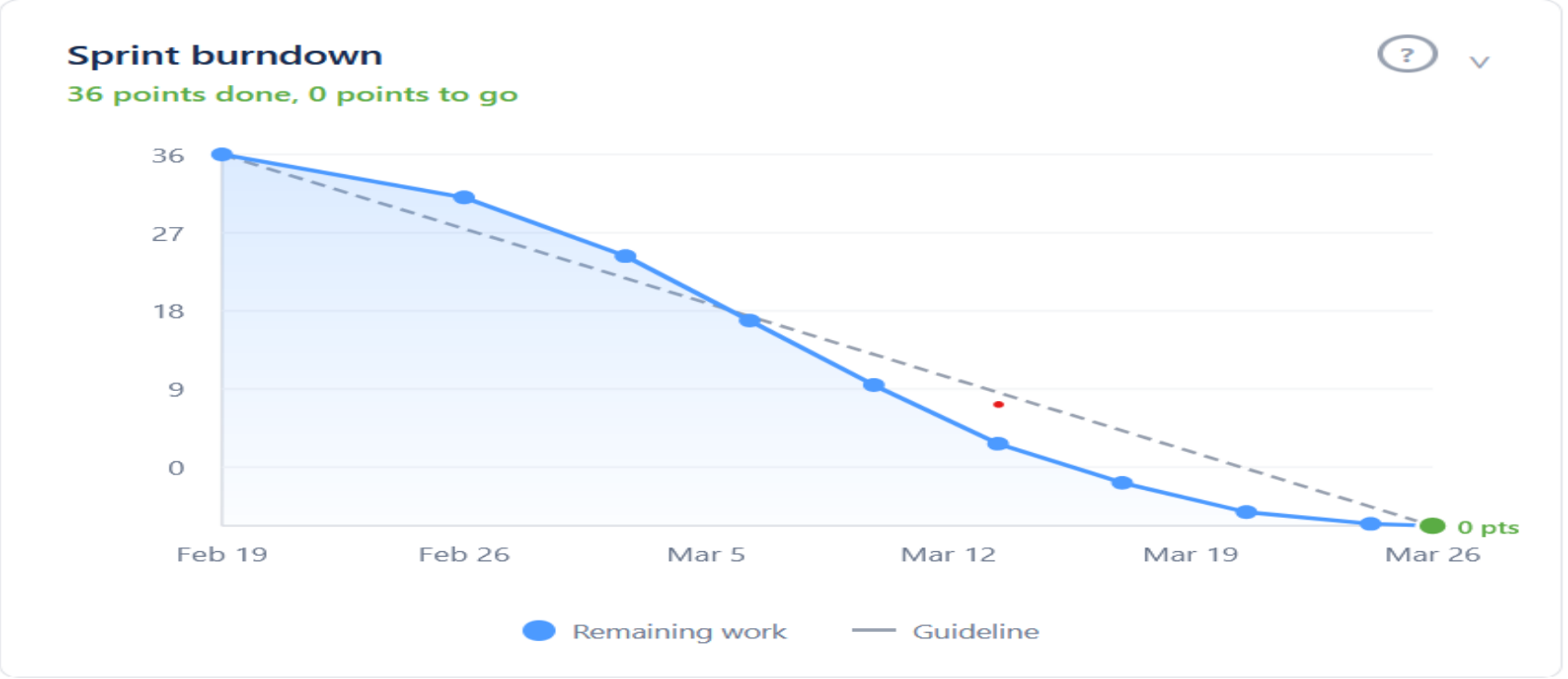
ID	Story	Points	Status
PB-09	Docker Compose + Schema Migrations	5 SP	DONE
PB-03	Row-Level Security Policies	3 SP	DONE
PB-08	Zod Validation Middleware	2 SP	DONE
PB-01	Tenant Registration + API Key Auth	3 SP	DONE
PB-02	POST /v1/notifications Endpoint	5 SP	DONE
PB-04	Async BullMQ Processing	5 SP	DONE
PB-07	Static Routing Logic	3 SP	DONE
PB-05	Email Delivery (Nodemailer/Mailtrap)	3 SP	DONE
PB-06	GET /v1/notifications/:id Status	2 SP	DONE
PB-10	Dashboard (React SPA)	5 SP	DONE

Team Velocity

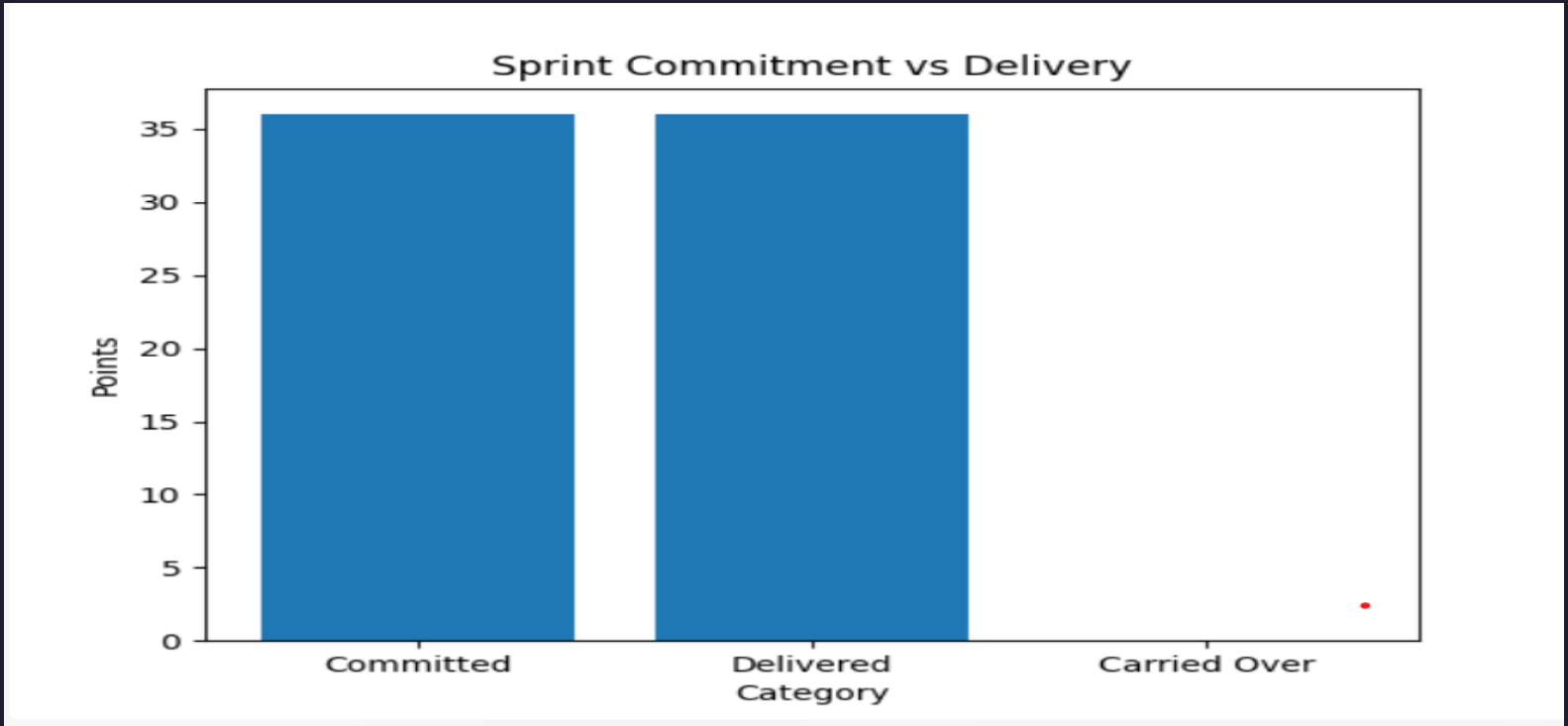
Total Estimated Points: 36
Total Points Delivered: 36
Total Points Carried Over: 0
Team Velocity: 36



Sprint 1 Burndown Chart



Completed / Committed Ratio



Sprint 1 Retrospective

What Went Well

- 8 PRs integrated into develop
- 14/14 acceptance tests passed
- Core endpoints working
- Dashboard live with real API data
- Docker Compose setup complete
- Proper Git workflow followed

What to Improve

- PR consistency issues (dotenv usage)
- TypeScript typing (avoid any)
- Duplicate endpoint implementation
- Field naming mismatch (camelCase vs snake case)
- Slack communication delays

Action Items

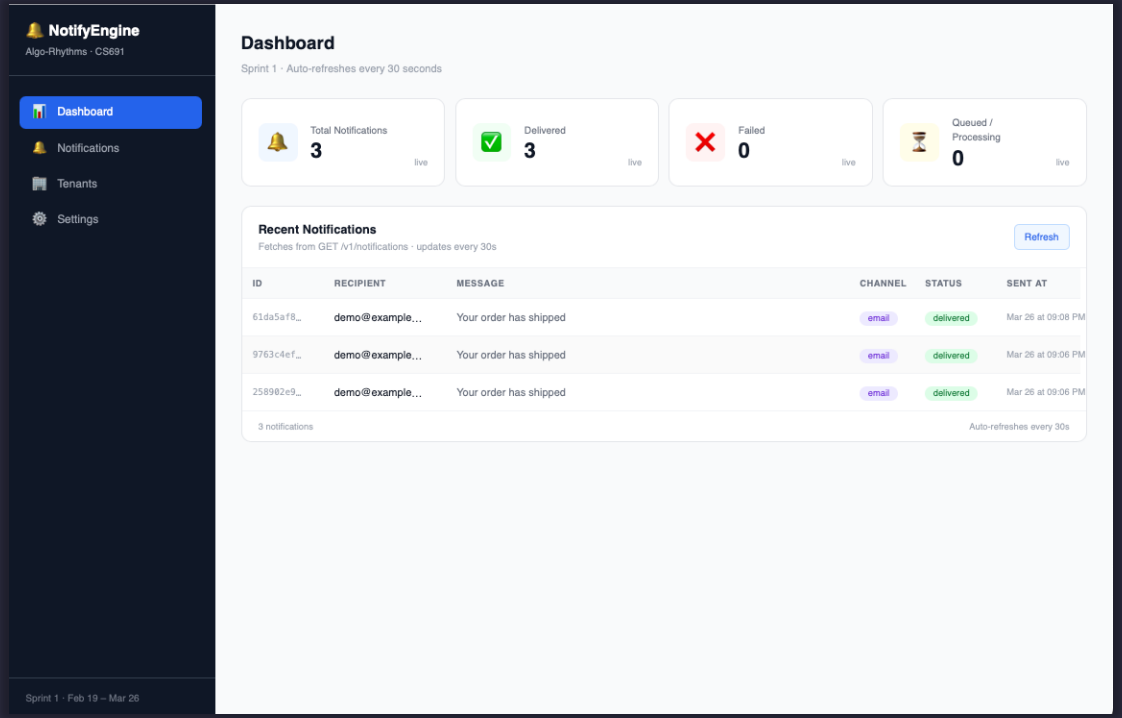
- PR self-review before submission
- No dotenv, no any, no hardcoded credentials
- Follow API contract and naming standards
- 30-min Sprint 2 kickoff code walkthrough
- Review existing codebase together
- Avoid duplicate work and improve coordination

Sprint 2 Planning

Mar 26 - Apr 16 | ML Integration

PB-11	XGBoost model training + prediction endpoints	5 SP
PB-12	Epsilon-greedy exploration (90/10 split)	3 SP
PB-13	Explore/exploit endpoint	3 SP
PB-14	Feature vector computation + storage	3 SP
PB-15	Model promotion pipeline (AUC-ROC gated)	3 SP
PB-16	Dashboard: Routing Intelligence page	3 SP
PB-17	WebSocket delivery channel	3 SP
CARRY	*None - 36/36 delivered*	0 SP

Project Demo - App Screenshots



Project Demo - API

Method	Endpoint	Description	Status
POST	/v1/tenants/register	Register tenant, returns API key	201
POST	/v1/notifications	Create notification, enqueue job	202
GET	/v1/notifications/:id	Notification status + delivery attempts	200
GET	/v1/notifications	List with cursor pagination	200
GET	/v1/notifications/summary	Counts by status	200
GET	/v1/engagement/track?nid=	Tracking pixel, records engagement	200

API Screenshots

Step 1: Register a tenant

```
● Bash(curl -s -X POST http://localhost:3000/v1/tenants/register \
      -H "Content-Type: application/json" \
      -d '{"company_name": "Acme Notifications"}' | python3 -m json.tool)
  └ {
        "tenant_id": "87119152-7222-4dc5-bcf3-89efdc416379",
        "company_name": "Acme Notifications",
        "slug": "acme-notifications",
        "api_key": "ne_test_9539429c9a2862e2ba52e5aee5eed1d84146f83e522fe4653892f6e26784d693",
        "message": "Tenant registered successfully. Save this API key now - it will not be shown again.",
        "created_at": "2026-03-27T01:04:51.718Z",
        "request_id": "a1a3c6be-2357-40e9-a0ee-1df0d6d20d30"
    }
```

Step 2: Send a notification

```
● Bash(curl -s -X POST http://localhost:3000/v1/notifications \
      -H "Content-Type: application/json" \
      -H "Authorization: Bearer ne_test_9539429c9a2862e2ba52e5aee5eed1d84146f83e522fe4653892f6e26784d693" \
      -d '{
        "recipient": "demo@example.com",
        "subject": "Your order has shipped",
        "body": "Order #4821 is on its way. Track it here.",
        "body_html": "<h1>Order Shipped</h1><p>Order #4821 is on its way.</p>",
        "priority": "high",
        "routing_mode": "adaptive"
      }' | python3 -m json.tool)
  └ {
        "id": "61da5af8-96f2-47da-bfa3-41187bab5be3",
        "status": "queued",
        "priority": "high",
        "routing_mode": "adaptive",
        "created_at": "2026-03-27T01:08:40.506Z",
        "status_url": "/v1/notifications/61da5af8-96f2-47da-bfa3-41187bab5be3",
        "request_id": "739266cc-ef5d-4c84-ac0a-d00bd2c74f37"
    }
```

Step 3: GET notification status

```
● Bash(sleep 5 && curl -s http://localhost:3000/v1/notifications/61da5af8-96f2-47da-bfa3-41187bab5be3 \
      -H "Authorization: Bearer ne_test_9539429c9a2862e2ba52e5aee5eed1d84146f83e522fe4653892f6e26784d693" | python3 -m json.tool)
  └ {
        "id": "61da5af8-96f2-47da-bfa3-41187bab5be3",
        "status": "delivered",
        "recipient": "demo@example.com",
        "subject": "Your order has shipped",
        "priority": "high",
        "routing_mode": "adaptive",
        "delivered_via": "email",
        "delivered_at": "2026-03-27T01:08:40.743Z",
        "failed_at": null,
        "routing_decision": null,
        "metadata": {},
        "created_at": "2026-03-27T01:08:40.506Z",
        "updated_at": "2026-03-27T01:08:40.743Z",
        "delivery_attempts": [
            {
                "channel_type": "email",
                "attempt_number": 1,
                "status": "success",
                "status_code": 200,
                "error_message": null,
                "engaged": null,
                "engagement_type": null,
                "engaged_at": null,
                "started_at": "2026-03-27T01:08:40.526Z",
                "completed_at": "2026-03-27T01:08:40.741Z",
                "duration_ms": 215
            }
        ],
        "request_id": "c7da536e-439b-415b-a24c-aee33ca61f6c"
    }
```

GitHub

https://github.com/htmw/2026S-Algo-Rhythms/wiki

Monorepo	Turborepo + npm workspaces
Branches	main / develop / feature branches
CI/CD	GitHub Actions (lint, test, security scan)
Structure	apps/ (api, worker, ml-service, dashboard) + packages/ (shared) + infra/

Live Application Demo

Docker Compose local stack demonstration

1. Register tenant via POST /v1/tenants/register
2. Send notification via POST /v1/notifications
3. Verify email arrives in Mailpit
4. Check status via GET /v1/notifications/:id
5. Simulate email open via tracking pixel
6. Verify engagement recorded in delivery attempt
7. View dashboard with live stats and notification table
8. Prove tenant isolation (Rival Corp gets 403)

Thank You

Questions?

Algo-Rhythms | NotifyEngine | notifyengine.io